

Sistemas de Informação | AMF
Inteligência Artificial II
14 de setembro de 2026

Aula 6 - Interpretação de modelos, algoritmos de clustering

Rhauani Fazul
rhauani.fazul@amf.edu.br

Interpretação de modelos

Por que interpretar modelos de IA?

Interpretação de modelos

Por que interpretar modelos de IA?

- Transparência e confiança em modelos complexos (“caixa-preta”).
- Diagnóstico e *debugging* de modelos.
- Explicabilidade em áreas reguladas (saúde, finanças, governo).
- Ética, responsabilidade e *accountability* em IA.

Interpretação de modelos

- **Cenário:** Modelo de crédito preditivo (aprovado/não aprovado).
- **Instância analisada:** João solicitou um cartão de crédito.
- **Resultado do modelo:** Negado.

Interpretação de modelos

- **Cenário:** Modelo de crédito preditivo (aprovado/não aprovado).
- **Instância analisada:** João solicitou um cartão de crédito.
- **Resultado do modelo:** Negado.
- **Explicação (exemplo):**
 - Renda baixa: $-0,35$ (peso negativo).
 - Histórico de dívidas: $-0,25$.
 - Idade: $+0,10$.
 - Tempo de emprego: $+0,15$.

Interpretação ???

Interpretação de modelos

- **Cenário:** Modelo de crédito preditivo (aprovado/não aprovado).
- **Instância analisada:** João solicitou um cartão de crédito.
- **Resultado do modelo:** Negado.
- **Explicação (exemplo):**
 - Renda baixa: $-0,35$ (peso negativo).
 - Histórico de dívidas: $-0,25$.
 - Idade: $+0,10$.
 - Tempo de emprego: $+0,15$.

Interpretação:

O modelo negou o crédito principalmente devido à renda e dívidas.

Se esses fatores fossem diferentes, o resultado poderia mudar.

Interpretação de modelos

Interpretabilidade global vs local

- Global: explica comportamento médio do modelo (quais variáveis mais pesam).
- Local: explica uma previsão específica (por que este caso foi classificado assim?).
- Complementares: visão geral + explicações pontuais.

Interpretação de modelos

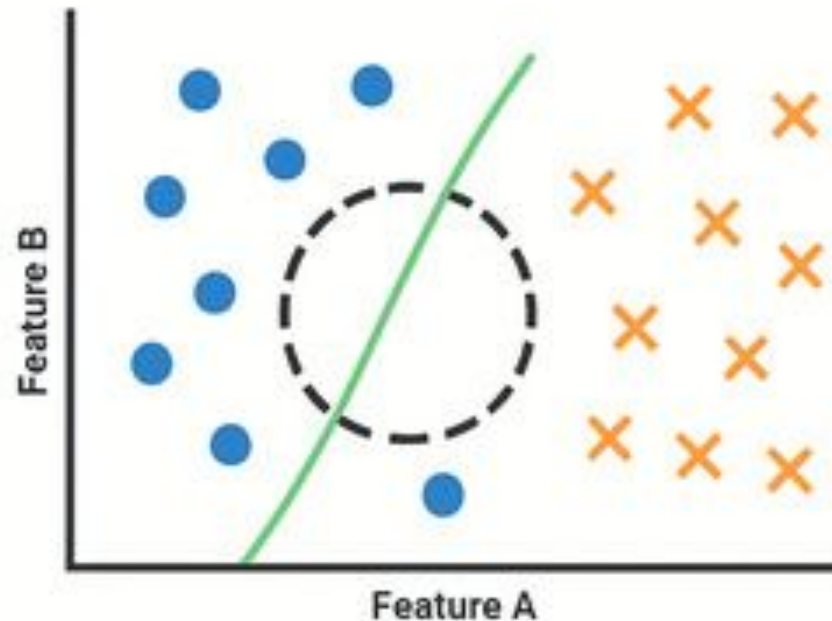
LIME

- “*Local Interpretable Model-agnostic Explanations*”.
- Cria pequenas perturbações nos dados próximos à instância analisada.
- Ajusta um modelo linear simples para aproximar o comportamento do modelo.
- Mostra as variáveis mais relevantes naquela previsão.

Interpretação de modelos

LIME

- **Cenário:** Previsão de sobrevivência no Titanic.
- **Objetivo:** explicar por que um passageiro foi classificado como “não sobreviveu”.



Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

1. Escolhe a instância a ser explicada:

```
# Treinamento modelo  
rf = RandomForestClassifier(n_estimators=200, random_state=42)  
rf.fit(X_train, y_train)  
  
# Escolhe uma instância para explicar  
i0 = 10  
x0 = X_test.iloc[[i0]]  
  
print("Previsão do modelo:", rf.predict(x0)[0],  
      "| Probabilidade de sobreviver:", rf.predict_proba(x0)[0,1])
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

2. Gera perturbações em torno desse passageiro (dados similares).

Imagine que você tem um passageiro específico do Titanic (ex.: homem, 35 anos, 3ª classe, tarifa £7).

O LIME cria cópias artificiais desse passageiro, mudando um pouco os valores (um pouco mais de idade, uma tarifa um pouco maior, outro porto de embarque etc.).

Esses pontos simulados ajudam a observar como o modelo reage a pequenas variações nos dados desse indivíduo.

É como “fazer perguntas” ao modelo: e se esse passageiro tivesse pago mais caro? e se fosse um pouco mais velho?

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

2. Gera perturbações em torno desse passageiro (dados similares).

```
# Criar perturbações ao redor do passageiro
n_samples = 1500
noise = np.random.normal(0, X_train.std(axis=0)*0.5, size=(n_samples,
X_train.shape[1]))
Z = x0.values + noise
Z = np.clip(Z, X_train.min().values, X_train.max().values)
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

3. Pergunta ao modelo original (ex.: Random Forest) as probabilidades para esses novos dados.

```
Z_proba = rf.predict_proba(Z)[: ,1]
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

4. Aplica pesos maiores às amostras mais próximas.

Nem todas as perturbações são realistas.

Se você criar um passageiro “muito diferente” do original (ex.: mulher, 1ª classe, 10 anos), isso já não representa bem o caso que você está explicando.

Por isso, o LIME dá mais importância (peso) às amostras mais parecidas com o passageiro original.

Assim, a explicação é realmente local, focada naquele caso específico.

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

4. Aplica pesos maiores às amostras mais próximas.

```
# Pesos pela proximidade  
dists = cosine_distances(Z, x0)  
kernel_width = np.sqrt(X_train.shape[1])  
weights = np.exp(-(dists**2)/(kernel_width**2))
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

5. Treina um modelo linear simples para explicar a decisão local.

O modelo original (ex.: Random Forest, XGBoost, Rede Neural) é complexo e difícil de interpretar.

- Ao redor de um único ponto (o passageiro), podemos aproximar o comportamento do modelo com algo mais simples, como uma regressão linear.
- Os coeficientes desse modelo linear local indicam como cada variável contribuiu para a previsão.

É como usar uma lupa: no mundo todo o terreno é cheio de curvas, mas num pedacinho pequeno você pode “aproximar por uma linha reta”.

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

5. Treina um modelo linear simples para explicar a decisão local.

```
# Ajustar modelo linear local
scaler = StandardScaler().fit(Z)
Z_std = scaler.transform(Z)

ridge = Ridge(alpha=1.0)
ridge.fit(Z_std, Z_proba, sample_weight=weights.ravel())
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

6. Explique pelos coeficientes: positivo aumenta a probabilidade da classe prevista; negativo diminui.

```
# Importâncias locais
coefs = ridge.coef_

importance = pd.Series(coefs, index=X_train.columns).sort_values(key=lambda s: abs(s),
ascending=False)

print("\nImportância local (LIME-style):")
print(importance.head())

# 8) Gráfico
importance.head(6).plot(kind="barh", figsize=(6,4))
plt.title("Explicação local (LIME-style) para o passageiro")
plt.show()
```

Interpretação de modelos

Ver: `1_lime.py`

Passos do LIME:

6. Explique pelos coeficientes: positivo aumenta a probabilidade da classe prevista; negativo diminui.

Previsão do modelo: 1 (sobreviveu, classe positiva) | Probabilidade de sobreviver: 0.66

fare	0.106903	→ pagar mais caro aumenta a chance de sobreviver (coeficiente positivo).
pclass	-0.088496	→ estar em classes mais baixas diminui a chance de sobreviver.
age	-0.072538	→ maior idade reduz a chance de sobreviver.
sex_male	-0.028656	→ ser homem reduz a chance de sobreviver.
embarked_Q	-0.002788	→ ter embarcado no porto Q tem um efeito levemente negativo.

Esse tipo de explicação permite dizer: “O modelo concluiu que o passageiro tinha 66% de probabilidade de sobreviver, principalmente porque pagou uma tarifa relativamente alta. Esse fator positivo compensou efeitos negativos de ser homem, mais velho e de classe baixa.”

Interpretação de modelos

SHAP

- “*SHapley Additive exPlanations*”.
- Inspirado na Teoria dos Jogos (valores de Shapley).
- Mede a contribuição de cada feature para a previsão.
- Propriedade: consistência e justiça na atribuição.
- Mais custoso computacionalmente, mas fornece explicações mais estáveis.

Interpretação de modelos

Valores de Shapley

- Vêm da Teoria dos Jogos Cooperativos (Shapley, 1953).
- Imagine que cada variável (feature) é um jogador de um time.
- O “jogo” é a previsão do modelo.
- Os valores de Shapley dizem quanto cada jogador contribuiu para o resultado, de forma justa:
 - Considera todas as combinações possíveis de features entrando no jogo.
 - Distribui o resultado proporcionalmente à contribuição marginal de cada variável.

Interpretação de modelos

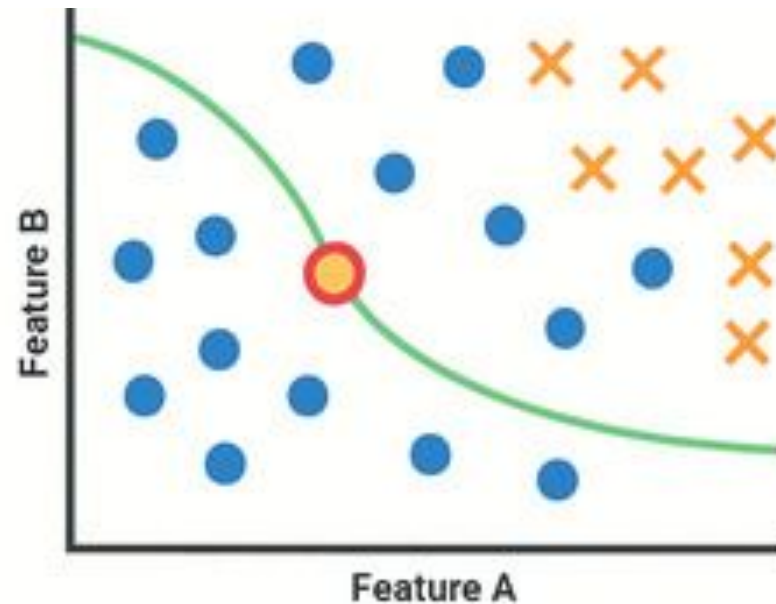
SHAP

Na prática em IA: cada previsão é decomposta em base value + soma das contribuições (SHAP values).

- *Base value*: previsão média do modelo em todo o dataset (ex.: probabilidade média de sobrevivência = 0.38).
- *SHAP values*: contribuição de cada feature para mover essa média até a previsão do passageiro específico.

Interpretação de modelos

SHAP



Sempre gera valores locais.

Mas pode ser agregado → vira também uma medida global.

Interpretação de modelos

Ver: `1_shap.py`

SHAP

Previsão do modelo: 1 (sobreviveu, classe positiva) | Probabilidade de sobreviver: 0.66

sex_male	0.267881
pclass	-0.118442
age	0.097280
embarked_S	0.030612
embarked_Q	-0.004372

- Destoa do resultado do LIME, onde ser homem e mais velho eram fatores negativos;
- Contraintuitivo para o problema Titanic, onde a gente espera que ser homem, mais velho e de classe baixa reduza a chance de sobreviver;
- O SHAP está dizendo que ser homem e mais velho aumentaram a chance de sobreviver nesse caso.

Interpretação de modelos

Ver: `1_shap.py`

SHAP

Não olha para média geral: ele olha para como essa instância específica afetou a saída do modelo, comparado ao valor médio da base.

Ele está dizendo: *“Esse passageiro teve 66% de chance de sobreviver. O principal motivo foi o modelo aprender que homens como ele (com esse perfil completo) tendem a sobreviver mais.”*

Apesar do perfil tradicionalmente menos favorecido (homem, mais velho, classe mais baixa), o modelo aprendeu que nessa combinação específica de características, esses fatores contribuíram positivamente para a previsão de sobrevivência.

Mas por que?

Interpretação de modelos

Ver: `1_shap.py`

SHAP

O modelo (Random Forest) pode ter aprendido regras como:

“Homens com certa idade, tarifa e porto de embarque às vezes sobrevivem mais.”

Reflexão: seria isso um sinal do **overfitting** da implementação do modelo avaliado?

Quando o SHAP parece estranho, o problema pode não ser a explicação, mas o que foi aprendido.

Interpretação de modelos

LIME:

- Simples, rápido, *model-agnostic*.
- Explicações locais (aproxima com um modelo linear), mas podem variar.

SHAP:

- Base matemática sólida.
- Explicações consistentes e globais+locais.
- Mais pesado em tempo de execução.

LIME e SHAP podem dar respostas diferentes porque usam métodos distintos.

Diferenças nos sinais indicam que a interação entre variáveis pode influenciar a decisão final.

Mais duas ferramentas para a *toolbox* de vocês!

Interpretação de modelos conexionistas

- SHAP e LIME foram criados pensando em modelos supervisionados tabulares.
- Funcionam super bem aqui, porque cada feature tem significado direto (ex: idade, renda, nota).
- Explicações são intuitivas → peso da variável sobre a predição.

Mas e para redes neurais (visão, texto, tabulares complexos)?

Interpretação de modelos conexionistas

Também funcionam, mas exigem adaptações:

- DeepSHAP e DeepLIFT foram criados para redes neurais profundas.
- LIME pode ser aplicado criando perturbações nas entradas (pixels, palavras).
- Usado para entender por que a rede deu determinado output.

Exemplo: destacar pixels que levaram à classificação de uma imagem como “gato”.

Interpretação de modelos conexionistas

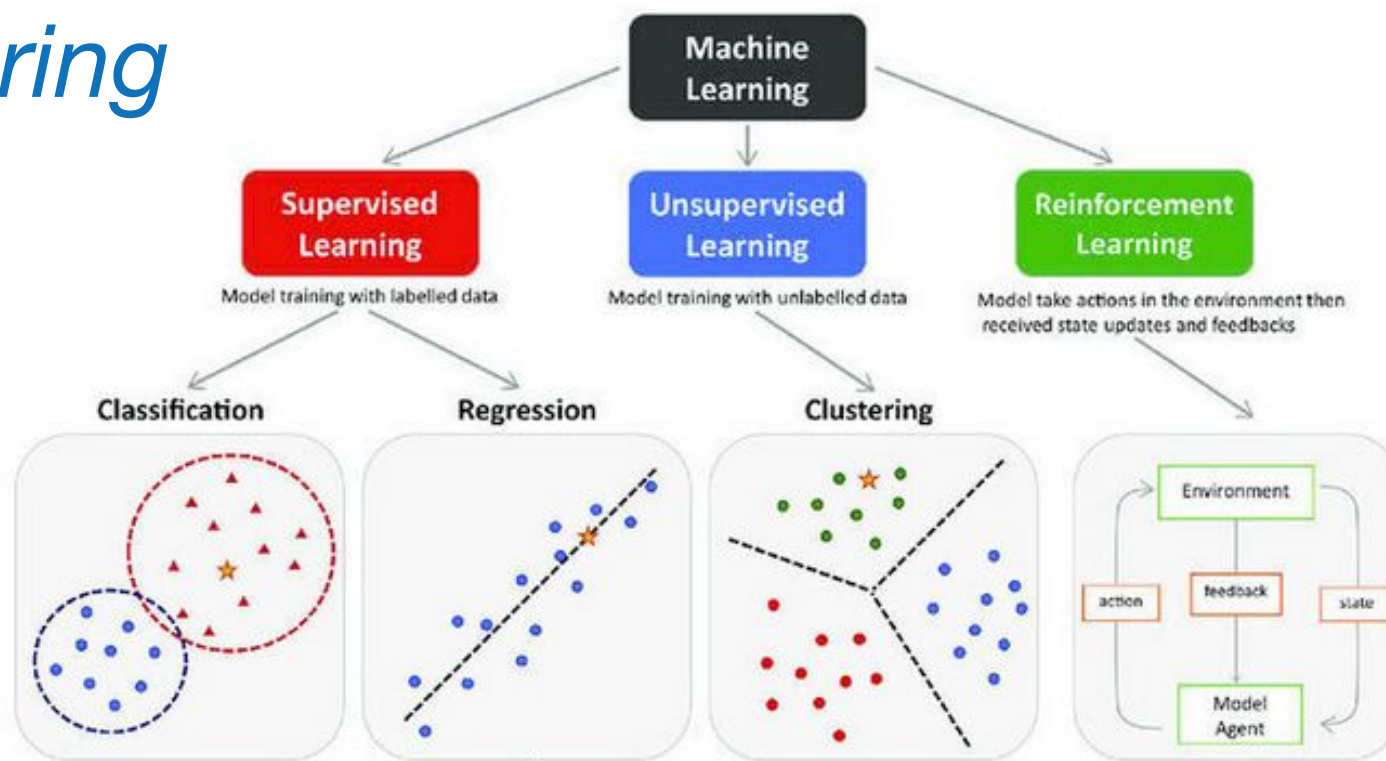
IA Generativa (LLMs, Diffusion Models, etc.)

- É mais desafiador, porque o espaço de entrada/saída é enorme.
- Ferramentas como SHAP e LIME ainda são usadas em casos específicos (ex: embeddings de texto), mas surgiram novas abordagens:
 - Attribution Maps (atenção em transformers).
 - Prompt interpretability (quais tokens mais pesaram na resposta).
 - Feature attribution em embeddings (usando SHAP sobre vetores de contexto).
- Área muito ativa de pesquisa → “*Explainable AI for Generative Models*”.

Clustering

O que é *clustering* (agrupamento)?

Clustering



- Não há variável alvo definida a priori: **agrupamento (clustering)**
- Há variável alvo/dependente ordinal ou contínua: **regressão**
- Há variável alvo/dependente categórica: **classificação**
- Aprendizado por reforço

Tarefa

O problema que você quer resolver

Classificação, Regressão, Clustering

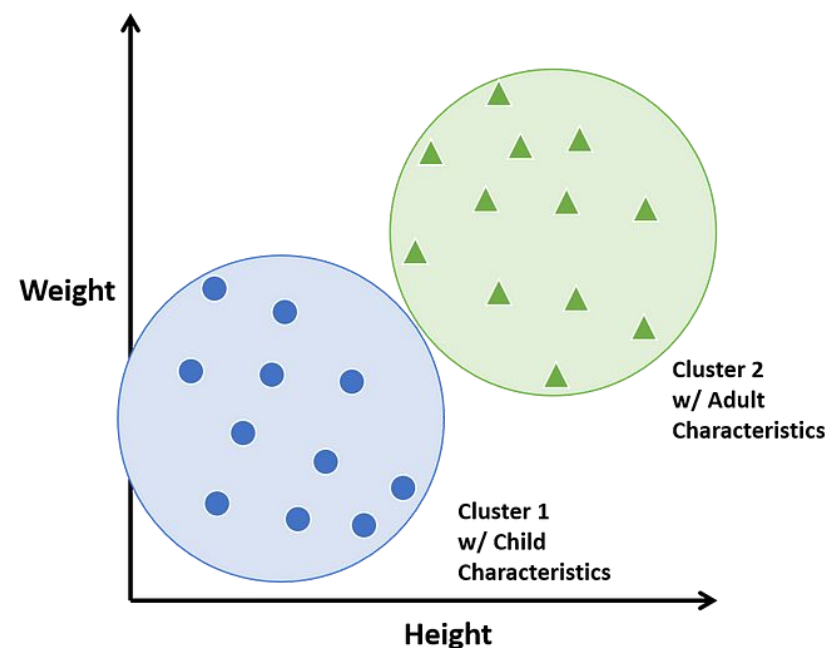
Modelo/Algoritmo

A ferramenta que você usa para resolver

Rede Neural, Árvore de Decisão, KNN, etc.

Clustering

- Técnica de **aprendizado não supervisionado**.
- **Agrupa dados semelhantes em “clusters”**:
 - Encontrar grupos (clusters) de dados similares.
- Muito usado para:
 - Exploração inicial de dados.
 - Segmentação de clientes.
 - Detecção de padrões ocultos.



Clustering

- Agrupar dados semelhantes/parecidos. Como definir semelhança?
Abordagens matemáticas são comumente adotadas
- Em geral, trata-se de um problema difícil:



Como encontrar textos semelhantes?



Clustering

- Noção de grupos pode ser ambígua:



Quantos?

Clustering

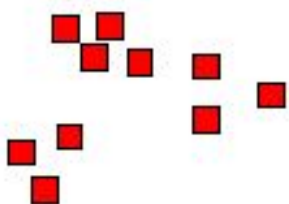
- Noção de grupos pode ser ambígua:



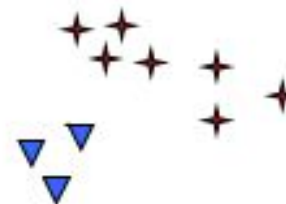
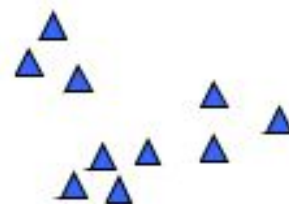
Quantos?



Seis grupos



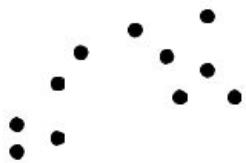
Dois grupos



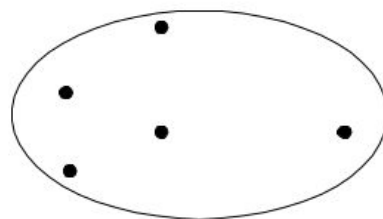
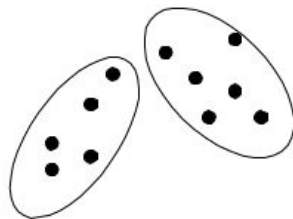
Quatro grupos



Clustering



Pontos originais



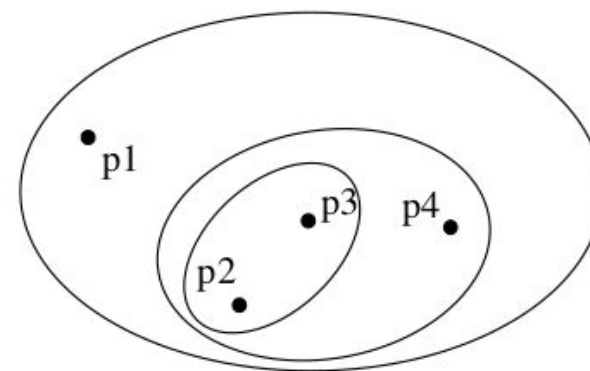
Agrupamento por partição

Partitional clustering

Uma divisão de objetos cujos grupos não se sobrepõem, de modo que cada objeto de dado está em somente um grupo.

Hierarchical clustering

Um conjunto de grupos alinhados em uma árvore hierárquica.



Agrupamento hierarquico tradicional

Prática!

Ver: `clustering_perfil.py`

Resposta o Questionário Perfil e vamos agrupar!

K-means

Agrupamento por centróides:

- **Objetivo:** dividir os dados em K grupos (clusters).
- Cada cluster é representado por um centróide (ponto médio).
- **Intuição:** minimizar a distância dentro do grupo → pontos ficam “mais próximos do seu centróide”.
- Limitação: precisa definir K antes e funciona melhor em clusters esféricos e de tamanhos semelhantes.

K-means

Ver: `1_kmeans.py`

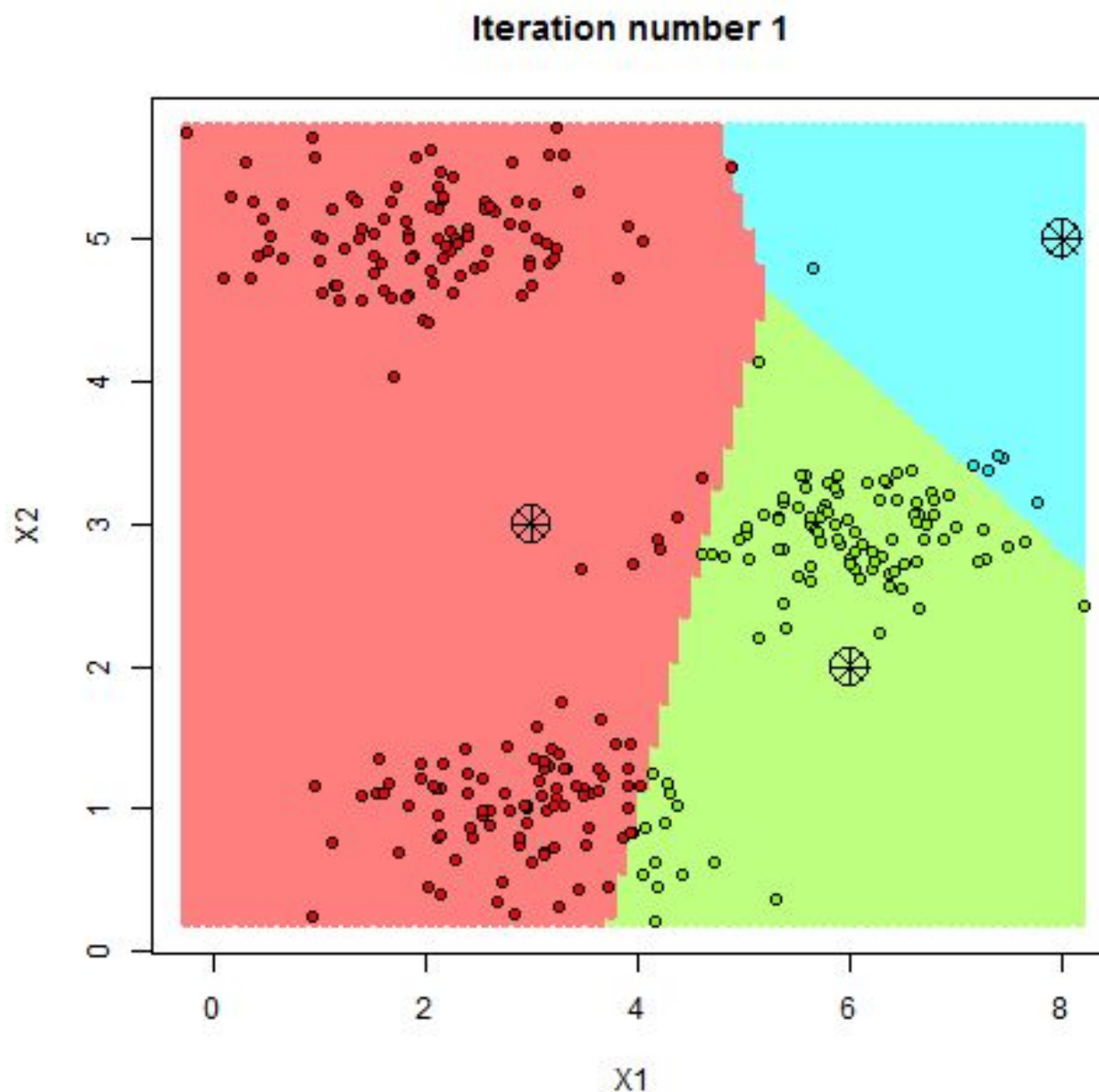
- Passos:
 - Escolhe-se K centróides iniciais (aleatórios).
 - Cada ponto é atribuído ao centróide mais próximo.
 - Recalcula-se os centróides como a média dos pontos em cada grupo.
 - Repete até os centróides pararem de mudar (convergência).

K-means

Algumas aplicações gerais:

- **Segmentação de clientes** em marketing (agrupamento de perfis de consumo).
- **Agrupamento de documentos** (ex.: separar notícias por tema).
- **Recomendação básica** (agrupar usuários/itens por similaridade).
- **Compressão de imagens** (redução de cores ao agrupar pixels semelhantes).

Ver: `2_kmeans.py`, `3_kmeans.py`

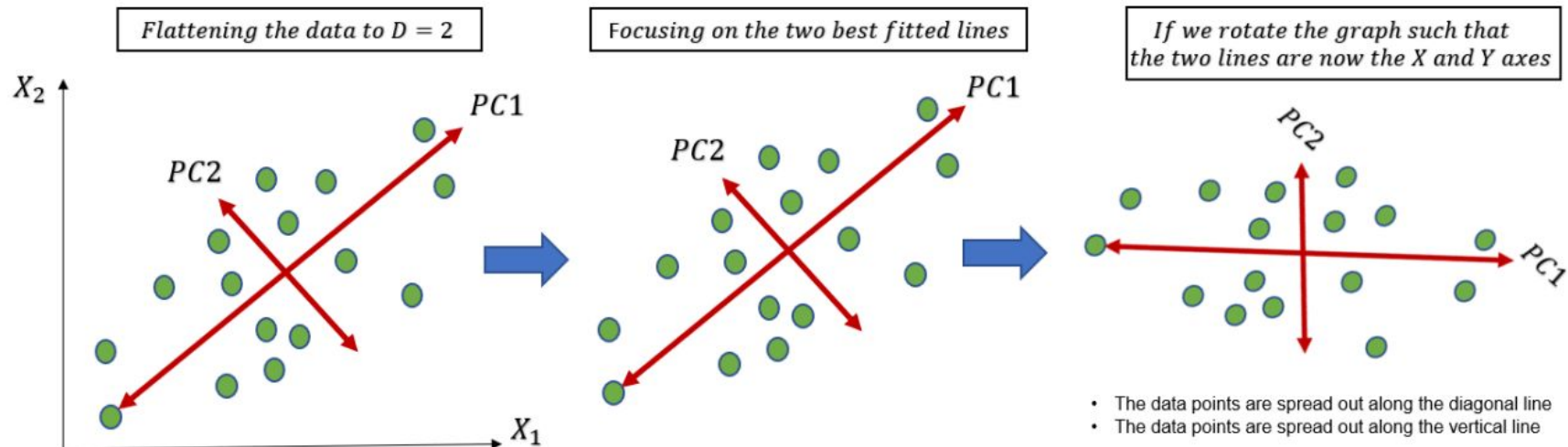
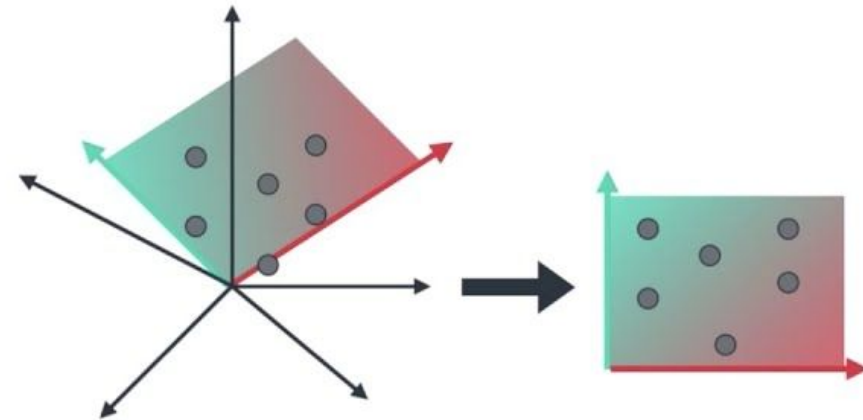
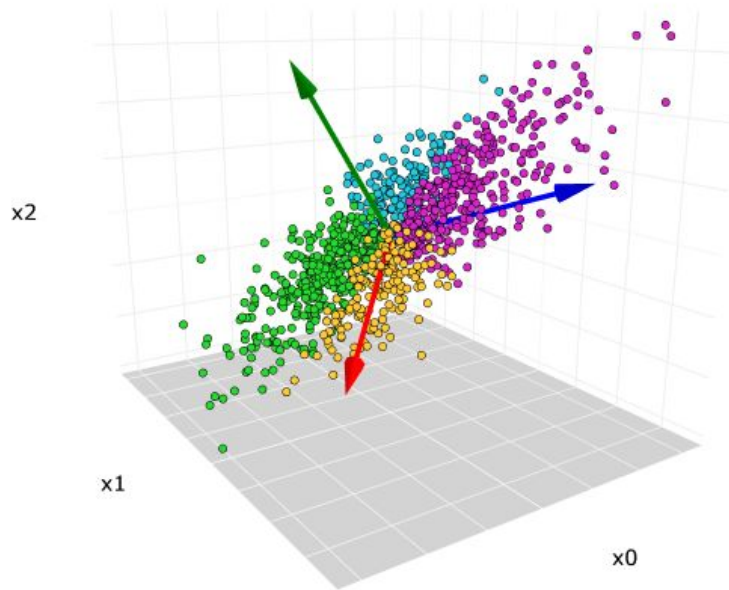


[Ref](#)

Principal Component Analysis (PCA)

- Técnica para reduzir a dimensionalidade de um dataset mantendo o máximo da informação possível.
- Intuição: conjunto de pontos em 3D.
 - O PCA encontra os eixos que melhor explicam a variação dos dados.
 - Projetamos os pontos nesses novos eixos → menos dimensões, mas dados ainda “fazem sentido”.
- Por que usar?
 - Facilita visualização (ex.: clusters em 2D).
 - Reduz ruído e redundância nos atributos.
 - Ajuda algoritmos a rodar mais rápido sem perder muita precisão.
- Transforma variáveis em combinações lineares → pode perder interpretabilidade direta.

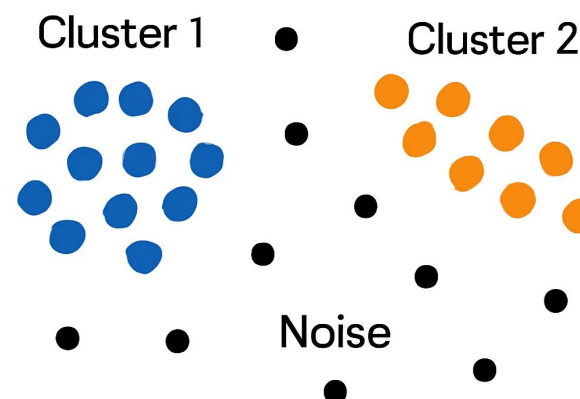
Principal Component Analysis (PCA)



DBSCAN

Density-Based Spatial Clustering of Applications with Noise:

- **Objetivo:** encontrar clusters com base em **densidade** de pontos.
- **Intuição:** “ilhas de densidade” viram clusters, áreas esparsas viram ruído.
- Vantagem: não precisa definir número de clusters K e detecta formas arbitrárias.



DBSCAN

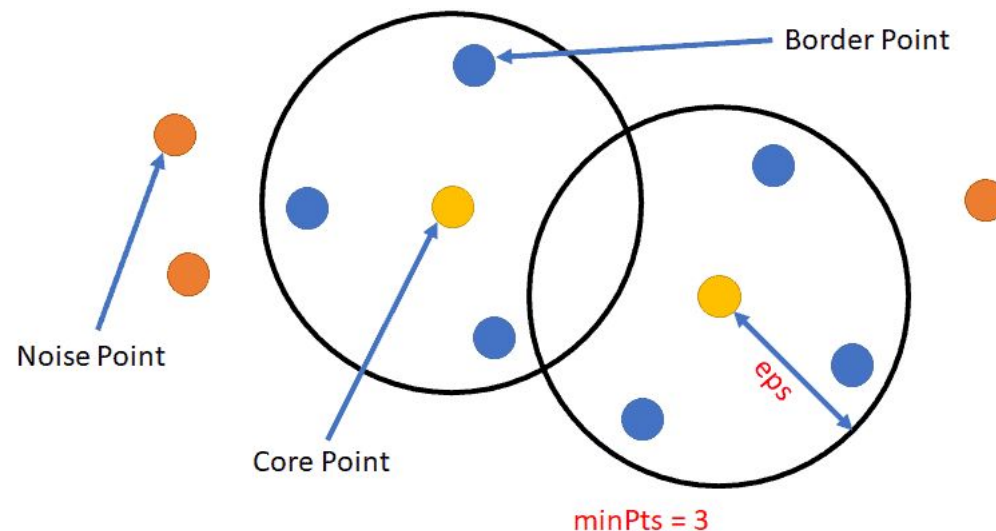
Ver: `1_dbscan.py`

- **Conceitos-chave:**

- ϵ (eps): raio de vizinhança.
- minPts: número mínimo de pontos para formar uma região densa.

- **Tipos de pontos:**

- *Core point*: tem pelo menos minPts vizinhos em ϵ .
- Border point: vizinho de um core, mas com poucos pontos em volta.
- *Noise (outlier)*: não pertence a nenhum cluster.

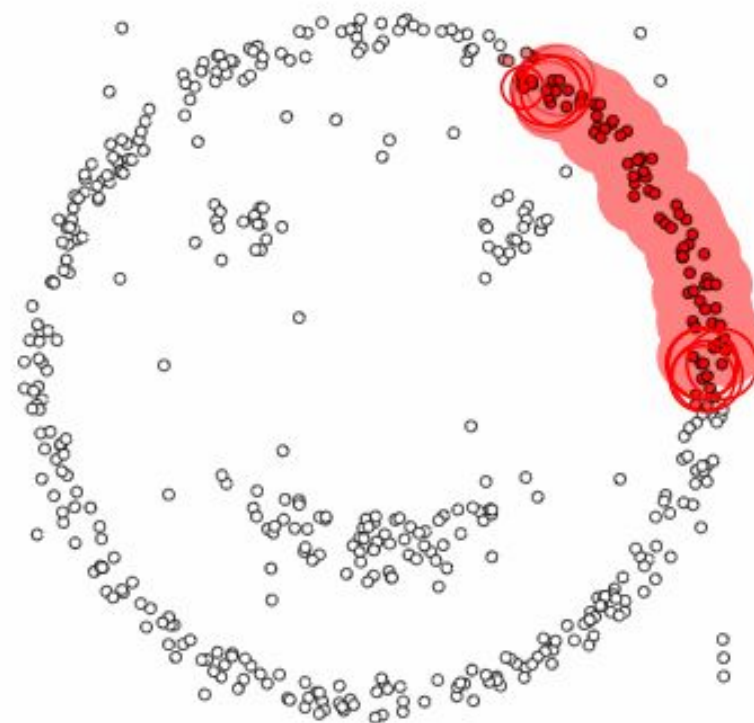


DBSCAN

Algumas aplicações gerais:

- **Detecção de fraudes e anomalias** em transações financeiras.
- **Agrupamento de pontos geográficos** (mapas, regiões de calor, GPS).
- **Reconhecimento de padrões em imagens** (ex.: formas irregulares).
- **Análise de tráfego de rede** (detectar ataques ou comportamento suspeito).
- **Ciência ambiental** (detectar áreas densas de poluição, espécies, etc.).

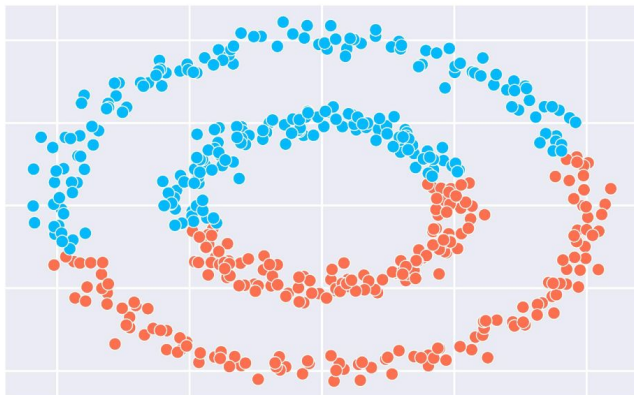
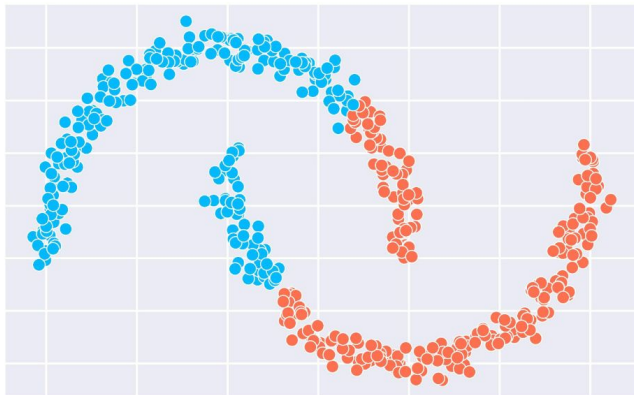
epsilon = 1.00
minPoints = 4



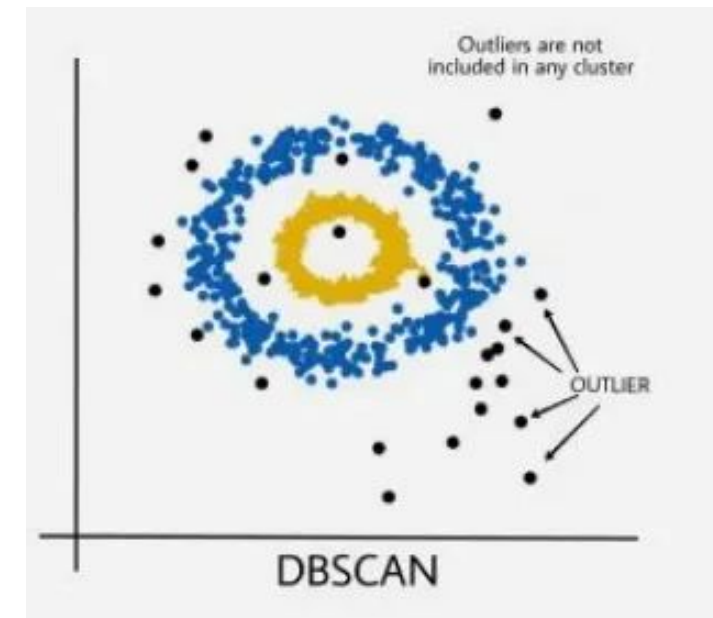
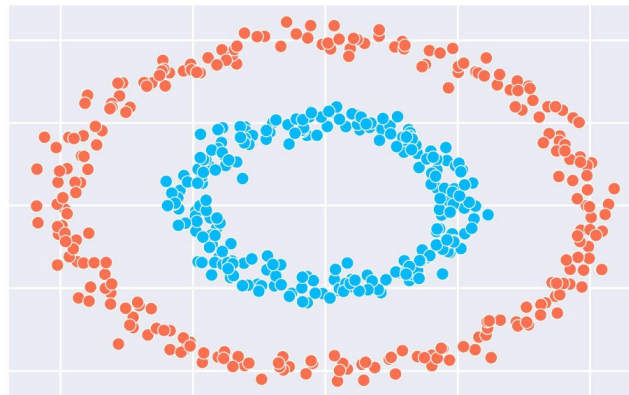
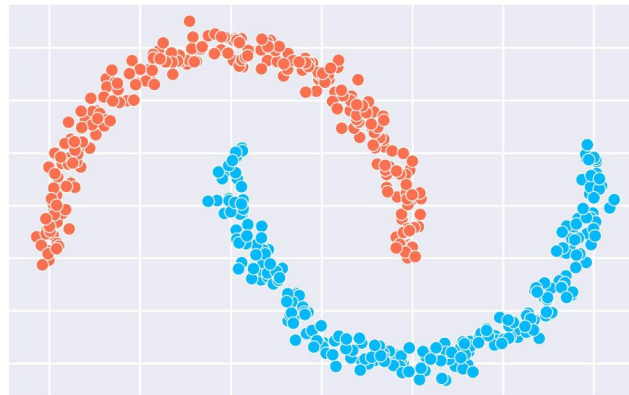
[Ref](#)

K-means vs DBSCAN

KMeans



DBSCAN



K-means vs DBSCAN

Critério / Algoritmo	K-means	DBSCAN
Número de clusters	Precisa definir K antes	Determinado automaticamente pela densidade
Forma dos clusters	Prefere clusters esféricos e semelhantes em tamanho	Detecta formas arbitrárias (curvas, alongados etc.)
<i>Outliers</i>	Sempre força o ponto a entrar em algum cluster	Identifica e rotula como ruído
Parâmetros	Apenas K	Precisa definir ϵ (raio) e minPts
Escalabilidade	Muito rápido, eficiente em grandes bases	Mais custoso, pode ser lento em grandes bases
Sensibilidade	Depende da escolha inicial dos centróides	Depende da escolha de ϵ e minPts
Dados de alta dimensão	Funciona razoavelmente bem	Perde eficiência e sentido de densidade

Reflexão

Por que estamos vendo esses algoritmos todos?

Reflexão

Por que estamos vendo esses algoritmos todos?

- **Base da IA** → tudo que existe hoje nasceu daqui.
- Ajudam a entender **como a máquina aprende e decide**.
- São usados na prática em várias áreas (não é só pra cumprir ementa).
- Ensinaaram a ciência de dados a lidar com erros, ruído, vieses e limites.
- Se você entende o clássico, fica muito mais fácil entender o avanço da IA nos dias de hoje (NN, DL, Gen. AI, ...)
- Em tecnologia, **quem domina a base consegue se adaptar ao que vier**.

Reflexão

O que você deve ter aprendido (no módulo de ML) até aqui:

- Comparar modelos clássicos → **conhecer e entender quando usar cada um.**
- Pré-processar dados e criar atributos → perceber que **dados bem tratados valem mais que algoritmos sofisticados.**
- Trabalhar com modelos tabulares e *boosting* → explorar técnicas aplicadas para datasets reais.
- (hoje) Interpretar modelos com SHAP e LIME → não só prever, mas **explicar decisões.**
- (hoje) Clustering (K-means, DBSCAN) → agrupar dados sem rótulo, entendendo intuição, prós e contras.

Visão crítica → nenhum algoritmo é “o melhor sempre”:
o valor está em **entender o contexto, escolher bem suas ferramentas, combinando técnicas** quando necessário!

Temas para pesquisa

- Semi-supervised learning
- Self-supervised learning
- Self-labeling / pseudo-labeling

Reflexão: possibilidade (não convencional)

- Usar aprendizado não supervisionado:
 - Os clusters não vêm com nomes de classes “oficiais”, mas você pode interpretá-los (ex.: cluster 1 = “clientes fiéis”, cluster 2 = “clientes ocasionais”).
- Transformar clusters em rótulos provisórios:
 - Esses rótulos não são “perfeitos”, mas já permitem tratar os dados como quase supervisionados.
- Aprendizado supervisionado em cima dos clusters:
 - O modelo supervisionado consegue generalizar para novos dados (não precisa rodar clustering toda vez).
 - Também permite usar métricas supervisionadas (acurácia, F1, etc.) para avaliar o desempenho.
- Atenção aos riscos:
 - Os rótulos derivados podem ser ruidosos. É importante validar os clusters antes de usá-los como verdade absoluta.
 - A interpretação dos clusters depende muito de conhecimento de domínio.

Atividade prática

- Incorporar um algoritmo de *clustering* em um *workflow* de IA.

Dicas para n8n:

Execute Command Node → `python3 file.py`

PythonFunction Node → seu código

Webhook Node → Server → seu código

Próxima aula...

- Avaliação de clusters.
- Sistemas de recomendação.